

Camino del Inca

Guía de instalación para empezar a programar

Cómo dejar tu PC con Windows lista, bajar el repositorio y levantar el proyecto. Son los pasos que ya se siguieron en una PC del grupo el 03/10/2026, con los problemas que aparecieron y cómo se resolvieron. Lleva entre 40 minutos y una hora.

Antes de empezar

- **Aceptá la invitación al repositorio.** Te llegó por correo y también está en github.com/notifications.
- **Los comandos se escriben en Git Bash** (se instala en el paso 2), salvo los que dicen PowerShell. Cada cuadro de comandos indica dónde va.
- **En Git Bash se pega con Shift + Insert** (o clic derecho → Paste). Ctrl + C no copia: corta el comando que está corriendo.
- **Seguí el orden.** Si un paso no da el resultado que dice, andá a la Parte 3 antes de seguir.
- **Para copiar los comandos** también podés abrir la misma guía en el repositorio: [docs/instalacion-windows.md](#) y [README.md](#).

QUÉ SE INSTALA	PARA QUÉ	PASO
Git	Control de versiones. Trae la terminal Git Bash	2
Node.js	Compila los estilos y scripts de las pantallas	3
PostgreSQL	La base de datos	4
PHP	El lenguaje en el que está hecho el sistema (Laravel)	5
Composer	Baja los paquetes de PHP que usa el proyecto	6

PARTE 1 Preparar la PC

1 Desactivar el Control inteligente de aplicaciones

Windows 11 trae una protección que bloquea los programas sin firma digital. El PHP oficial para Windows no está firmado, así que con esa protección activada **PHP deja de arrancar**, a veces después de haber funcionado un rato. No se puede hacer una excepción para un solo programa. Fue lo que más tiempo nos llevó: conviene hacerlo primero.

1. Abrí **Seguridad de Windows** → **Control de aplicaciones y navegador** → **Configuración de Control inteligente de aplicaciones**.
2. Elegí **Desactivado**. Si ya figura así, no hay que hacer nada.

El antivirus de Windows y SmartScreen siguen funcionando. Con Windows actualizado se puede volver a activar más adelante; antes de confirmar, leé el aviso de esa pantalla. En Windows 10 esta protección no existe: salteá el paso.

2 Git

Si ya tenés Git Bash, salteá la instalación. Si no: bajá el instalador de git-scm.com y dejá todas las opciones por defecto.

Después abrí **Git Bash** y configurá tu nombre y tu correo, para que cada commit quede a tu nombre (la cátedra pidió que programemos todos):

```
GIT BASH
git config --global user.name "Nombre Apellido"
git config --global user.email "el-correo-de-tu-cuenta-de-github"
```

3 Node.js

Bajá el instalador **LTS** para Windows de nodejs.org y dejá las opciones por defecto.

4 PostgreSQL

Bajá el instalador de [postgresql.org/download/windows](https://www.postgresql.org/download/windows) y ejecutalo.

- Te pide una contraseña para el usuario `postgres`: **anotala**, la vas a usar en el paso 10.
- Dejá el puerto `5432`.
- Al final, destildá "Stack Builder".

Comprobá que el servidor quedó funcionando:

```
GIT BASH
powershell -Command "Get-Service *postgres*"
```

Tiene que mostrar: una línea con el estado `Running`. Si no muestra nada, el instalador no creó el servicio: resuelvelo con el problema B de la Parte 3.

5 PHP

1. En windows.php.net/download elegí PHP 8.4 u 8.5 → **x64 Non Thread Safe** → **Zip**.
2. Descomprimilo en `C:\php`, de modo que quede `C:\php\php.exe` (no una carpeta dentro de otra).
3. En Git Bash, creá la configuración y habilitá las extensiones que usa el proyecto:

```
GIT BASH
cd /c/php
cp php.ini-development php.ini
sed -i -E 's/^(extension_dir = "ext")/\1/' php.ini
sed -i -E 's/^(extension=(curl|fileinfo|intl|mbstring|openssl|zip))\b)/\1/' php.ini
sed -i -E 's/^(extension=(pdo_pgsql|pgsql|pdo_sqlite|sqlite3))\b)/\1/' php.ini
./php -m | grep -E "intl|pgsql|zip|mbstring"
```

Tiene que mostrar: `intl`, `mbstring`, `pdo_pgsql`, `pgsql` y `zip`.

6 Composer

Bajá y ejecutá getcomposer.org/Composer-Setup.exe. Cuando pregunte qué PHP usar, elegí `C:\php\php.exe` y aceptá que lo agregue al PATH. El resto, por defecto.

7 Comprobar que quedó todo

Cerrá **todas** las ventanas de Git Bash y abrí una nueva: los programas recién instalados sólo se reconocen en una terminal abierta después de instalarlos.

```
GIT BASH
php -v
composer -V
node -v
```

Tiene que mostrar: la versión de cada uno: PHP 8.4 u 8.5, Composer 2 y Node 22 o superior.

PARTE 2 Bajar y levantar el proyecto

8 Clonar el repositorio

Elegí dónde vas a tener el proyecto (por ejemplo `C:\Proyectos`), abrí esa carpeta en el Explorador, clic derecho → **Open Git Bash here**, y corré:

```
GIT BASH
git clone https://github.com/Tomacods/camino-del-inca.git
cd camino-del-inca
```

Si se abre el navegador pidiendo que inicies sesión en GitHub, hazelo: es para que Git pueda subir tus cambios.

9 Instalar las dependencias del proyecto

```
GIT BASH
composer install
npm install
```

Tarda unos minutos. `composer install` termina con una lista larga de archivos de Filament y `npm install` puede mostrar advertencias (*warn*): las dos cosas son normales.

10 Configurar el proyecto y crear la base

a. Creá tu archivo de configuración (es de cada uno y no se sube al repositorio):

```
GIT BASH
cp .env.example .env
php artisan key:generate
```

b. Creá una base de datos vacía llamada `camino_del_inca`. Con pgAdmin: Servers → PostgreSQL → clic derecho en **Databases** → Create → Database... O con este comando, cambiando `18` por tu versión de PostgreSQL:

```
GIT BASH
"/c/Program Files/PostgreSQL/18/bin/createdb" -U postgres camino_del_inca
```

Pide la contraseña del paso 4 (no se ve mientras la escribís). Si vuelve sin mostrar nada, la base quedó creada.

c. Poné esa contraseña en la configuración:

```
GIT BASH
notepad .env
```

Buscá la línea `DB_PASSWORD=`, completala (por ejemplo `DB_PASSWORD=1234`, sin espacios), guardá y cerrá.

11 Crear las tablas y tu usuario del panel

```
GIT BASH
php artisan migrate --seed
```

Tiene que mostrar: una lista de migraciones, cada una terminada en `DONE` .

```
GIT BASH
php artisan make:filament-user --name="Tu nombre" --email="admin@caminodelinca.test" --password="admin1234"
```

El correo y la contraseña son sólo para entrar al panel en tu PC; podés poner otros.

12 Levantar la aplicación

Hacen falta **dos ventanas** de Git Bash abiertas en la carpeta del proyecto. En la primera:

```
GIT BASH · VENTANA 1
php artisan serve
```

En la segunda:

```
GIT BASH · VENTANA 2
npm run dev
```

Dejá las dos abiertas y probá en el navegador:

DIRECCIÓN	QUÉ TENÉS QUE VER
<code>http://localhost:8000</code>	La página de bienvenida de Laravel
<code>http://localhost:8000/admin</code>	El inicio de sesión del panel. Entrás con el correo y la contraseña del paso 11 y aparece el "Escritorio" con el título <i>Camino del Inca</i>

Si ves las dos, terminaste. Avisá en el grupo. Para apagar la aplicación, Ctrl + C en cada ventana. Cada vez que te sientes a trabajar, repetí sólo este paso.

PARTE 3 Si algo falla

A Los comandos `php` no muestran nada

Es el Control inteligente de aplicaciones (paso 1). Puede pasar aunque PHP haya funcionado antes. Para confirmarlo:

```
GIT BASH
php -v; echo "codigo=$?"
```

Si responde sólo `codigo=127`, es eso: desactivó la protección como indica el paso 1 y probá de nuevo. Si sigue igual, reiniciá la PC.

B *Connection refused* en el puerto 5432, o no existe el servicio de PostgreSQL

El instalador copió los programas pero no creó la base inicial ni el servicio (la carpeta `data` de la instalación queda vacía). Se completa a mano, sin reinstalar. Abrió **PowerShell como administrador** (menú Inicio → PowerShell → clic derecho → Ejecutar como administrador) y corré, cambiando `18` por tu versión en las dos líneas donde aparece:

```
POWERSHELL COMO ADMINISTRADOR
cd "C:\Program Files\PostgreSQL\18\bin"
.\initdb.exe -D ..\data -U postgres -W -A scram-sha-256 -E UTF8 --locale=C --locale-provider=icu --icu-locale=es-AR
.\pg_ctl.exe register -N postgresql -D "C:\Program Files\PostgreSQL\18\data"
Start-Service postgresql
Get-Service postgresql
```

La segunda línea pide dos veces la contraseña nueva del usuario `postgres`. La última tiene que mostrar `Running`; el servicio queda configurado para arrancar solo con la PC.

C *could not find driver* al correr `php artisan migrate`

PHP no tiene habilitada la extensión de PostgreSQL. Repetí los tres comandos `sed` del paso 5 y comprobá con:

```
GIT BASH
php -m | grep pgsql
```

D *password authentication failed for user "postgres"*

La contraseña de `DB_PASSWORD` en el archivo `.env` no es la de tu PostgreSQL. Corregila (paso 10 c) y volvé a correr el comando.

E *command not found* justo después de instalar algo

Cerrá todas las ventanas de Git Bash y abrí una nueva. Si sigue sin encontrar `php`, revisá que exista `C:\php\php.exe` y volvé a ejecutar el instalador de Composer (paso 6), que es el que agrega PHP al PATH.

F La página muestra *Vite manifest not found*

Falta la segunda ventana: corré `npm run dev` en la carpeta del proyecto y recargá la página.

G Error de `VCRUNTIME140.dll` al ejecutar PHP

Falta el "Visual C++ Redistributable 2015-2022 x64" de Microsoft. Se baja gratis del sitio de Microsoft, se instala y se vuelve a probar.

H Aviso "Parte de esta aplicación se ha bloqueado" al abrir pgAdmin

También es el Control inteligente de aplicaciones. Normalmente pgAdmin abre igual; si no, creá la base con el comando del paso 10 b.

Si no es ninguno de estos: copió el mensaje de error completo y la última línea que escribiste, y pasalo al grupo. No sigas con los pasos siguientes hasta resolverlo.

PARTE 4 Cómo trabajamos

El detalle está en `CONTRIBUTING.md`, en la raíz del repositorio. Lo esencial:

1. `main` siempre funciona. Nadie sube directo a `main`: todo entra por *pull request*.
2. Una rama por caso de uso (o por arreglo). Ramas cortas, que se integran en días y no en semanas.
3. Otro integrante revisa y aprueba el *pull request* antes de integrarlo.
4. Una migración que ya está en `main` no se edita: se crea otra que la modifique.
5. Cada viernes: versión estable etiquetada y respaldo de la base.

El ciclo de cada tarea

```
GIT BASH
git switch main
git pull
git switch -c cu-14-realizar-reserva
```

Programás, probás, y vas guardando con commits chicos:

```
GIT BASH
git add -A
git commit -m "CU-14: retiene el cupo al pasar al pago"
git push -u origin cu-14-realizar-reserva
```

Después abris el *pull request* en GitHub, otro integrante lo prueba y lo aprueba, y se integra a `main`.

Después de cada `git pull` que trae cambios

```
GIT BASH
composer install
npm install
php artisan migrate
```

Qué no se sube nunca

El archivo `.env`, credenciales de Mercado Pago o de Mailtrap, las carpetas `vendor` y `node_modules`, y los respaldos de la base. Ya están excluidos: si Git ofrece subir alguno, algo está mal.

Dónde mirar

ARCHIVO DEL REPOSITORIO	QUÉ TIENE
<code>docs/reparto-casos-de-uso.md</code>	Quién hace cada caso de uso y en qué orden
<code>docs/esquema-base-de-datos.md</code>	Las 15 tablas del Documento de Normalización
<code>docs/convenciones-de-codigo.md</code>	Nombres, dónde va cada cosa y formato del código
<code>docs/instalacion-windows.md</code>	Esta misma instalación, para copiar los comandos